NYU TANDON SCHOOL OF ENGINEERING

Computer Science and Engineering

<Your Project Name>

Software System Design Specification (SSDS)

Version 2.1

Document Number: SSDS-001

Project Team Number: A/B <number>

Project Team Members: (name and NET_ID)

REVIEW AND APPROVALS

Printed Name and Title	Function (Author, Reviewer, Approval)	Date	Signature
Professor Strauss	Author	<Date>	

REVISION LEVEL

Date	Revision Number	Purpose
January 20, 2026	Version 1.0	Initial Release
January 27, 2026	Version 2.0	Major Update Corrected defects Added Design Sections
February 24, 2026	Version 2.1	Defects Fixed

TABLE OF CONTENTS

1. DOCUMENT PURPOSE 5

1.1 PURPOSE 5

2. INTRODUCTION 5

2.1 SCOPE 5
2.2 IDENTIFICATION 5
2.3 BOUNDS 6
2.4 OBJECTIVES 6
2.5 CONTEXT DIAGRAM 6
2.6 ADDITIONAL DESCRIPTIVE ITEMS 6

3. GLOSSARY 7

4. REFERENCE DOCUMENTS 7

5. BUSINESS REQUIREMENTS 7

5.1 TECHNOLOGY 7
5.2 ECONOMICS 7
5.3 REGULATORY AND LEGAL 7
5.4 MARKET CONSIDERATIONS 8
5.5 RISKS AND ALTERNATIVES 8
5.6 HUMAN RESOURCES AND TRAINING 8

6. USER REQUIREMENTS (DESCRIPTIVE, FUNCTIONAL, AND NON-FUNCTIONAL REQUIREMENTS) 9

6.1 FUNCTIONAL DESCRIPTIVE DETAILED REQUIREMENTS 9
6.2 NON-FUNCTIONAL DESCRIPTIVE DETAILED REQUIREMENTS 9

7. SYSTEM ARCHITECTURE 9

8. DETAILED SYSTEM REQUIREMENTS – USE CASES 10

9. SYSTEM MODEL (UML) 11

9.1 STATIC-CLASS DIAGRAMS 11
9.2 DYNAMIC - BEHAVIORAL MODELS 12

10. SYSTEM-WIDE DESIGN DECISIONS 12

10.1 SOFTWARE COMPONENT ARCHITECTURAL DESIGN 12
10.2 SOFTWARE ARCHITECTURE GENERAL DESCRIPTION 12
10.3 SOFTWARE ITEM COMPONENTS 12
10.4 COMPONENT INTERFACE IDENTIFICATION 13
10.5 SOFTWARE COMPONENT CONCEPT OF EXECUTION 13

11. SOFTWARE ITEM DETAILED DESIGN 13

11.1 STRUCTURE 13
11.2 STATIC RELATIONSHIP OF SOFTWARE UNIT 13
11.3 BEHAVIOR 13
11.4 CONCEPT OF EXECUTION 14
11.5 INTERFACE DESIGN 14

12. DEPLOYMENT ARCHITECTURE 14

12.1 PHYSICAL DEPLOYMENT ARCHITECTURE DIAGRAM 14

13. EVOLUTION OF THE SSDS 14

14. RATIONALE 15

15. NOTES 15

16. APPENDICES 15

16.1 SYSTEM TEST PLAN REQUIREMENTS 15
16.2 QUALIFICATION PROVISIONS 15
16.3 REQUIREMENTS TRACEABILITY 16
16.4 SCHEDULE TRACKING 16
16.5 DEFECT TRACKING 18
16.6 DICTIONARIES 19

17. INDEX 19

1. DOCUMENT PURPOSE

1.1 Purpose

This subsection describes the purpose of this document.

a. Delineate the purpose of this SSDS
    a. Requirements, Analysis, and Design
b. Specify the intended audience of the SSDS

2. INTRODUCTION

This subsection should include:

a. Identify the system to be produced
b. Refer to and state the results of the needs in the form of a brief expression of the customer's problem(s). Explain what the system will do to meet those needs.
c. Describe the application of the system specified. As a part of this, it should describe all relevant top-level benefits, objectives, and goals as precisely as possible
d. Be consistent with the system proposal if it exists
e. Product perspective – puts the product into perspective concerning other products if a product is truly a standalone system (explicitly stated). If it is part of a larger system, the interface to that system is described here.

2.1 Scope

Identify functionality that is included in the system and functionality that is NOT included in the system. It must be consistent with the Context Diagram.

2.2 Identification

Identification of this document (name, number, revision)

2.3 Bounds

System boundaries - derived from the scope statement in the System Project Management Plan

2.4 Objectives

Including project priority, type of delivery, Life cycle (single deliverable, incremental, or evolutionary), and initial deliverable (milestone) dates.

2.5 Context Diagram

Appropriate diagrams and narratives are used to provide an overview of the system's context and to define all significant interfaces crossing its boundaries.

2.6 Additional Descriptive Items

a. Product functions – provide a summary or list of the primary functions that will perform (for example, an accounting system performing accounts receivable and general ledger functionality)
b. User characteristics – describe those general characteristics of the product's intended users, including experience, knowledge, and technical expertise. It should not state specific requirements, but instead, provide the rationale why specific requirements are later specified
c. Constraints – provide a general description of any items limiting the developer’s options. These **could** be based on policies and procedures and include:
    1. Regulatory
    2. Hardware limitations
    3. Interface to other applications
    4. Operations
    5. Audit functions
    6. Control functions
    7. Development language
    8. Communication protocols
    9. Priority
    10. Safety and security

d. Assumptions and dependencies – list each factor affecting the requirements. These factors are not designed constraints; any changes can affect the requirements. (For example, the specification of a specific operating system on a particular hardware platform. If the OS or hardware were not available, the requirements would have to be modified.

e. Requirements subsets—For iterative development, identify requirements subsets and those that could be delayed until future releases.

3. GLOSSARY

This section should describe:

Define technical terms used in the document. It would be best if you did not make assumptions about the experience or expertise of the reader.

4. REFERENCE DOCUMENTS

Include a reference to your Project Proposal

Team Number, Document Name, Document Number, Version, Date

5. BUSINESS REQUIREMENTS

5.1 Technology

Business technology drivers for this system, if any

5.2 Economics

Business economic drivers for this system, if any

5.3 Regulatory and Legal

It may not be relevant to your project.

5.4 Market Considerations

Business market drivers for this system, if any

5.5 Risks and Alternatives

Identify system risks and alternative solutions to mitigate each risk

Risk Table list format (suggest using a table rather than a list format)

Risk Type: Business
Description:
Probability:
How discovered:
Responsible Party
Status:
Mitigation Plan:

Risk Type: Operational
Description:
Probability:
How discovered:
Responsible Party
Status:
Mitigation Plan:

Risk Type: Technology:
Description:
Probability:
How discovered:
Responsible Party
Status:
Mitigation Plan:

Risk Type: Economic
Description:
Probability:
How discovered:
Responsible Party
Status:
Mitigation Plan:

5.6 Human Resources and Training

Business (project) resources and training required

6. USER REQUIREMENTS (DESCRIPTIVE, FUNCTIONAL, AND NON-FUNCTIONAL REQUIREMENTS)

This section should contain all software requirements in sufficient detail to enable designers to design a system that satisfies them and testers to test the system to meet them. Users, operators, or external systems should perceive every requirement externally.

The following principles should be applied:

a. Specific requirements should be stated in conformance with all the characteristics described in earlier documents (for example, the proposal) or earlier sections of the document
b. specific requirements should be cross-referenced (traceable) to earlier documents that they relate to
c. all requirements should be uniquely identifiable
d. Careful attention should be given to organizing the requirements for readability and understandability

6.1 Functional Descriptive Detailed Requirements

Functional requirements definition

Set of functional requirement specifications for each definition

(Must have 6 sets of definitions and specifications)

6.2 Non-Functional Descriptive Detailed Requirements

Non-Functional requirements definitions

See Lecture Slides for taxonomy categories and details.

7. SYSTEM ARCHITECTURE

This section should present a high-level overview of the anticipated system architecture, showing how functions are distributed across system modules. Reused architectural components should be highlighted.

8. DETAILED SYSTEM REQUIREMENTS – USE CASES

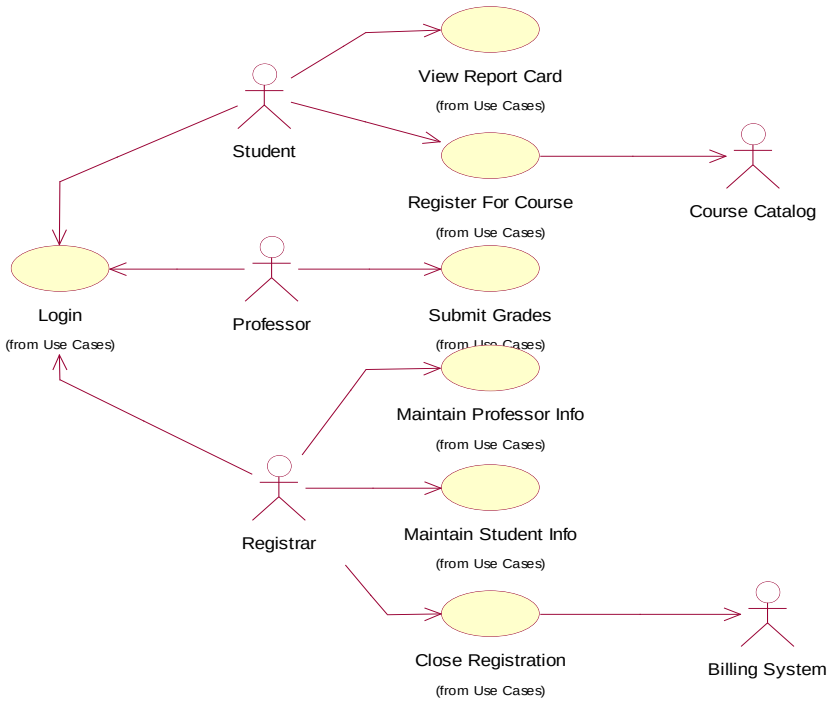
8.1. Requirement Use Cases

This subsection contains complete Use Cases, ranging from high-level views to low (decomposed) levels. Every use case must be included in a Use Case diagram and have a respective Use Case description.

Req1 - Login

8.1.1 Use Case Diagrams

Below is a sample use case diagram. Include the boundary rectangles in your diagrams.



8.1.2 Use Case Descriptions

Use Case Name		
Description		
Pre-Conditions		
Flows	**Basic or Normal Flows**	1. 2. 3.
	Alternative Flows	1. 2. 3.
Post Conditions		
Special Requirements		
Extension Points		

9. SYSTEM MODEL (UML)

Graphical system model

9.1 Static-Class Diagrams

Candidate classes (list)

Class Diagram (without interaction)

Class Interaction Diagrams (class interaction, relationship, and cardinality)

9.2 Dynamic - Behavioral Models

State diagrams (One for each Use Case/Functional Requirement)

Activity diagrams (One for each Use Case/Functional Requirement)

Sequence Diagrams (One for each Use Case

Requirement – each entity class must be included in at least one Sequence Diagram – interface classes must be on at least one Sequence Diagram)

10. SYSTEM-WIDE DESIGN DECISIONS

Before beginning the section, update all Functional Requirements (Use Case, Use Case Diagrams), Non-Functional Requirements, and Domain Requirements.

This section can be referred to as the Process Architecture

Collaboration Diagrams (One for the system flow)

10.1 Software Component Architectural Design

Using the updated set of Functional Requirements, Non-Functional Requirements, and Domain Requirements and the initial logical (component) architecture diagram from Analysis as a baseline, decompose the system into lower-level components.

Provide a detailed architecture diagram for the system component relationships and a detailed architecture diagram for **each** component.

10.2 Software Architecture General Description

Brief description of the overall architecture and rationale (why and how) for component decomposition

10.3 Software Item Components

Identify and describe the functionality of each component

10.4 Component Interface Identification

The interface between components must have a unique name and association (name, description, component1, component2). **Only the unique ID and associated attributes are included here. The detailed interface design is contained in section 4.**

10.5 Software Component Concept of Execution

Describe what motive and event causes a component to enter an execution state and the expected outcome of that component.

11. SOFTWARE ITEM DETAILED DESIGN

11.1 Structure

The final set of system elements.

Classes are created and modified to support the implementation environment.

11.1.1 Software Unit Detailed Design

Class diagrams for each unit (component).

Classes include boundary, interface, and other special classes.

11.2 Static Relationship of Software Unit

Class relationship (Interaction) diagrams with associations.

Key Event diagrams and key event dictionary

11.2.1 Run-time Object Instances

State diagrams for class methods.

Relationship between object instantiations – for example, threads, queuing mechanisms

11.3 Behavior

11.3.1 Sequence Diagrams

Complete set of sequence diagrams (between and within) components.

One per Use Case (Written Requirement Definition)

11.3.2 State Diagrams

Complete a set of class state diagrams.

11.3.3 Activity Diagrams

Complete set of activity diagrams.

11.4 Concept of Execution

Describe what motive and event causes a software unit to enter an execution state and the expected outcome of that execution.

11.5 Interface Design

Class Diagrams, Sequence Diagrams, State Diagrams, and Activity Diagrams
If applicable

11.5.1 Unique Identifier of Interface

If additional interfaces are identified during the decomposition, provide a unique identifier for each new interface (name, description, component1, component2).

11.5.2 Interface Identification and Diagrams

Provide a design specification for each interface identified for implementation in the system. If the interface is a reusable component, describe the implementation (for example, ODBC or SQL with its version number)

12. DEPLOYMENT ARCHITECTURE

12.1 Physical Deployment Architecture Diagram

13. EVOLUTION OF THE SSDS

This section describes how this document will evolve as the system development process progresses. It may be impossible to specify details when the project is initiated because we do not have all the knowledge required to do so. Additional charges may result from deficiencies, shortcomings, inaccuracies, or changes in the system environment.

A formal change process should be initiated to identify, control, track, and report projected changes. Approved changes should be incorporated into this document to ensure:

a. An accurate and complete trail of changes
b. Permit the review of current and superseded portions of the document

14. RATIONALE

15. NOTES

16. APPENDICES

16.1 System Test Plan Requirements

This section describes the SQA testing process, product testing, and acceptance testing.

Test plans with scenario testing and required simulators

16.2 Qualification Provisions

This section explains how this document will be reviewed for quality. Describes the verification and validation process that will be employed to verify the document's quality. Include the **four review types**. The documents should be:

a. Correct
b. Unambiguous

c. Complete
d. Consistent
e. Stable
f. Verifiable
g. Modifiable (malleable)
h. traceable

16.3 Requirements Traceability

This section describes how each requirement is traceable from its source to instances in each succeeding artifact. It must facilitate referencing each requirement in future development or enhancement documentation. Two types of traceability are forward (to all artifacts spawned by this document) and backward (to previous stages of development).

16.4 Schedule Tracking

Hours

Artifact or Deliverable	Who (individual or Team)	Estimated	Actual	Difference
SSDS - Domain	For each team member			
	Summary for the Entire Team			

Artifact or Deliverable	Who (individual or Team)	Estimated	Actual	Difference
SSDS - Requirements	For each team member			

	Summary for the Entire Team			

Artifact or Deliverable	Who (individual or Team)	Estimated	Actual	Difference
SSDS - Analysis	For each team member			
	Summary for the Entire Team			

Artifact or Deliverable	Who (individual or Team)	Estimated	Actual	Difference
SSDS - Design	For each team member			
	Summary for the Entire Team			

Cumulative

Artifact or Deliverable	Who (individual or Team)	Estimated	Actual	Difference
Cumulative - Summary	For each team member			
	Summary for the Entire Team			

16.5 Defect Tracking

Counts

Artifact or Deliverable	Who (individual or Team)	Estimated	Actual	Difference
SSDS - Domain	For each team member			
	Summary for the Entire Team			

Artifact or Deliverable	Who (individual or Team)	Estimated	Actual	Difference
SSDS - Requirements	For each team member			
	Summary for the Entire Team			

Artifact or Deliverable	Who (individual or Team)	Estimated	Actual	Difference
SSDS - Analysis	For each team member			
	Summary for the Entire Team			

Artifact or Deliverable	Who (individual or Team)	Estimated	Actual	Difference
SSDS - Design	For each team member			

	Summary for the Entire Team			

Cumulative

Artifact or Deliverable	Who (individual or Team)	Estimated	Actual	Difference
Cumulative - Summary	For each team member			
	Summary for the entire Team			

16.6 Dictionaries

See Data Dictionary – template (dictionary.pdf)

17. INDEX